



TEAM CHALLENGE · 55 MINUTES

Put the workstation *back in order*

Make the robot reliable, and show with measurements what your change brings.

The mission

PART	LOOKS LIKE	DESTINATION
bolt_box	red cube, 4.5 cm	top drawer
sensor	red cube, 4 cm	top drawer
bracket	blue bar, 8 × 3 × 3 cm	kitting tray
gauge	yellow cube, 5 cm	do not touch

FINAL STATE parts in their place, drawer closed (less than 2 cm open), gauge moved less than 2 cm, nothing on the floor. Before dropping a part into the drawer, open it at least 18 cm.

Read the destinations from `tools.rules()` instead of copying this table into your program.

Your first improvement

MEASURE in the VS Code terminal (menu Terminal, New Terminal: it opens in `~/vinci-workshop`) with `./evaluate.sh`. About 2 min for 8 episodes. Write down successes, score and simulated time.

CHANGE ONE THING in `team/config.yaml`: under `verification:`, set `enabled` to `true`, then `Ctrl+S`.

MEASURE AGAIN with the same settings, and read the score detail: `identified` goes from 0 to 10.

`final_state_verified` stays at 0: that one is yours to earn next.

EXPLAIN what changed: the final state, its verification, or both?

RUN	SUCCESSSES	SCORE	SIM TIME
Start			
Verification			
Your idea			

In rehearsal: 80 then 90 out of 100, with 8 successes out of 8 both times. The score moved, not the success.

Then, *one track* · 35 to 40 minutes

DISCOVERY Compare the drawer-opening providers in `config.yaml`, one at a time: `rl`, a trained policy, fast; `analytic`, a geometric controller; `visual`, a DAgger student that sees through the cameras, slower.

PROGRAMMING In `brain.py`, inspect after each action, compare with the expected result, retry if needed, with a limit on attempts and time. Check the final state.

FURTHER Handle parts and destinations from `tools.rules()`, find a failure in the traces, test on more episodes. A personal coding agent is optional: see `AGENTS.md`.

Measure *and* submit

Edit `team/brain.py` for decisions, `team/skills.py` for movements, `team/config.yaml` for settings. `solve(tools)` is called once per episode.

WHAT TO CALL IN YOUR BRAIN

<code>tools.rules()</code>	destinations and the preservation rule
<code>tools.inspect()</code>	parts, slots, drawer; privileged state allowed
<code>tools.call("open_drawer", provider="rl")</code>	open: <code>rl</code> , <code>analytic</code> or <code>visual</code>
<code>tools.call("pick_and_place", obj=..., target=...)</code>	place into <code>top_drawer</code> or <code>kitting_tray</code>
<code>tools.call("close_drawer")</code>	close the drawer
<code>tools.time_left(), tools.decide(...)</code>	time left, trace of your choices
<code>tools.lab.observe()["rgb_front"]</code>	camera images, when they run

A skill returns `ok`, `reason`, `duration_s`, `evidence`. **ok is its opinion: check the resulting state.** The rest of the API is in `team/README.md`.

SUCCESS AND SCORE Success describes the final state; points also count the steps. Budget: 240 simulated seconds per episode. The ranking keeps the best average score submitted on 8 identical DEV episodes, seeds 0 to 7, no video. Ties allowed.

THE RULES Change the team code, choose the skills, retrain the allowed models. Do not touch the simulator, the evaluator, the levels or the physics, and read the world through `tools` and `tools.lab.observe()`.

IF SOMETHING GOES WRONG `./reset.sh` resets the scene, `./reset.sh --team` restores the starter code and saves yours in `results/`. The **RESTART WORKSHOP** icon starts everything over, deleting nothing. Show any error message to the facilitator.

MEASURING A CHANGE

Choose **one option** to compute the score:

```
./evaluate.sh
no video · recommended · 1 to 2 min

./evaluate.sh --videos 2
optional · 9 to 10 min in our test
```

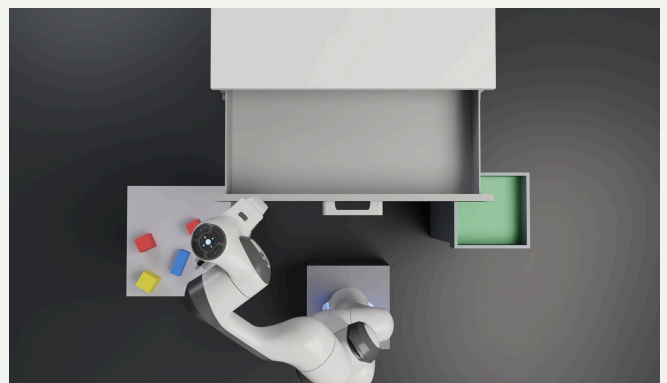
With video, the desktop may become less fluid; let it finish. You can skip videos and submit without any. Durations depend on settings and provider.

`./evaluate.sh --episodes 20`: more runs.

`./submit.sh`: submit the current version.

Read `results/latest/report.txt`, then the traces. Compare with identical settings and keep the path of each run: **latest** changes at every evaluation.

SCORE COMPONENT	POINTS
parts identified, grasped, correctly placed	10 · 15 · 25
no wrong placement, drawer handled	10 · 15
final state verified, task complete	10 · 15
total	100



The workstation seen from above: the parts on the work table, the drawer open in front of the robot, the kitting tray on its right.

BEFORE LEAVING

Save, run `./submit.sh`, then read its score with `./submit.sh --status`. The computation continues in the background. Several submissions allowed: the best valid score counts.

“We changed... ; our measurement shows... ; what remains to check is...”